



PROJECT PRESENTATION

CallFlow

A Sales Call Management System — assign leads, run calls, log outcomes, and watch every call roll up into live sales dashboards and targets.

PostgreSQL

Express

React

Node.js

Prisma ORM

JWT Auth

Tailwind CSS

Recharts

THE PROBLEM

Sales call teams run blind

- Leads are tracked in **spreadsheets and notebooks** — calls get missed, follow-ups slip.
- Managers have **no live view** of who is calling, what is converting, or who is behind target.
- Assigning customers to agents is **manual and messy**; nobody owns the lead.
- Monthly targets are guessed at, not measured.

THE SOLUTION

One loop, fully tracked

CallFlow turns the whole sales-call cycle into a single, auditable workflow.



TECH STACK

PERN + Prisma

Frontend

- React + Vite
- Tailwind CSS
- React Router
- Recharts (charts)
- Axios API client

Backend

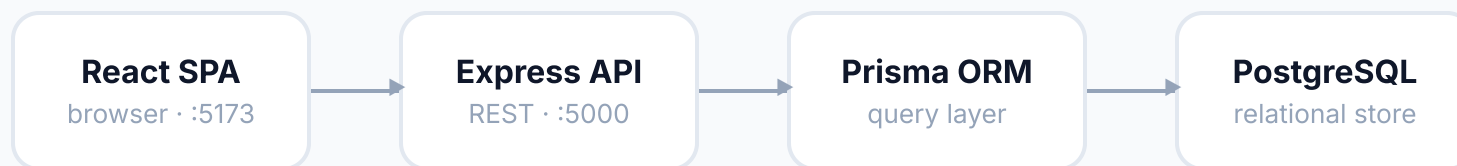
- Node.js + Express
- Prisma ORM
- Zod validation
- REST API under `/api`

Data & Auth

- PostgreSQL
- JWT (Bearer tokens)
- bcrypt password hashing
- Role-based access

ARCHITECTURE

How a request flows



- Vite dev server **proxies /api** to Express — no CORS pain in dev.
- Every protected route runs **authenticate** → **requireRole** middleware before the controller.
- Agents' queries are **automatically scoped** to their own customers and calls.

DATA MODEL

Three tables do the work

TABLE	KEY FIELDS	RELATIONS
User	name, email, passwordHash, role (ADMIN/AGENT), phone, monthlyTarget	has many Customers, has many Calls
Customer	name, phone, email, city, status, notes, lastCallAt, nextFollowUpAt	belongs to an agent (User), has many Calls
Call	status, notes, saleAmount, followUpAt, createdAt	belongs to Customer + Agent

Logging a call is one transaction: it inserts a **Call** *and* advances the customer's **status** + **lastCallAt**.

ACCESS CONTROL

Two roles, clear boundaries

Admin

- Full team dashboard & reports
- Create / edit / delete agents
- All customers — add, edit, delete, re-assign
- Company-wide call log
- Set monthly targets

Agent

- Personal dashboard & target progress
- Only **their** assigned customers
- Add customers (auto-assigned to self)
- Call Console — log every outcome
- No access to other agents' data

Run the whole floor

Dashboard

Total calls, total sales, active agents, team target %, a 30-day sales chart, top agents this month, and the latest calls.

Agents

Add / edit / delete. Per-agent monthly calls, sales and target progress. Drill into an agent for their chart + full call history.

Customers

Search & filter by status or agent. Inline re-assignment. Add / edit / delete. Last call and next follow-up at a glance.

Call Log

Every call across the team, filterable by agent and outcome, with notes and sale amounts.

Everything for the next call



My Dashboard

Calls today & this month, sales this month, a live target progress bar, and upcoming follow-ups.



My Customers

Only the leads assigned to me. Search, filter, and add new customers on the fly.



Call Console

Pick a customer, hit the `tel:` dial link, and log the result without leaving the screen.

HIGHLIGHT

The Call Console

- Searchable customer list on the left, live customer card on the right.
- One-tap outcome: **Interested** · **Callback** · **No Answer** · **Not Interested** · **Sale**.
- Choosing **Sale** reveals a required amount field.
- Optional **follow-up date** + free-text notes.
- On save: the call is stored and the customer's status & last-call time move forward automatically.



Why it matters

The agent never touches a spreadsheet. Every dial becomes structured data the moment it happens — which is exactly what the dashboards and targets are built on.

SECURITY

Built in, not bolted on

- **bcrypt** hashes every password — plain text is never stored.
- **JWT bearer tokens** signed with a server secret; short-lived and verified on every request.
- **Role middleware** (`requireRole('ADMIN')`) guards admin-only routes.
- **Ownership checks** — an agent physically cannot read or edit another agent's customers.
- **Zod validation** on every write; bad input is rejected with a clear 400.
- Secrets live in `.env` — **git-ignored**, never committed.

REPORTING

Numbers that update themselves

30d

Rolling sales & call series

Top 5

Agent leaderboard (month)

%

Target attainment per agent

6

Customer status buckets

All computed server-side with Prisma aggregates and a couple of grouped SQL queries — the client just renders what the dashboard endpoint returns.

RUN IT

From clone to login

```
# 1 - Backend
cd server
npm install
cp .env.example .env          # set DATABASE_URL + JWT_SECRET
npx prisma migrate dev        # create tables
npm run db:seed               # admin + 4 agents + demo data
npm run dev                   # API → :5000

# 2 - Frontend
cd client
npm install
npm run dev                   # app → :5173
```

ROADMAP

What comes next

- **Twilio Voice** — dial customers straight from the browser, record calls.
- **Socket.IO** — push live updates to the admin dashboard as calls land.
- **CSV import / export** for bulk customer lists.
- **Activity / audit trail** on every assignment and status change.
- Deploy: Vercel (client) + Render/Railway (API + Postgres).

THANK YOU

CallFlow

Sales Call Management System — PostgreSQL · Express · React · Node · Prisma

Source code & setup guide:

github.com/zaminaliabro/callflow